Lecture 5. Closures, and Functional Programming in Python and JavaScript

Programming Paradigms, Fall 2026 — 23 Sep 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. Pure Functions and Mutation
3. Closures
4. Python
5. JavaScript
6. Choosing a Representation

Where We Are

This Lecture

Last week: loops as `for` forms, values on demand, and `struct`.

This Lecture

Last week: loops as `for` forms, values on demand, and `struct`.

This week we leave Racket for Python and JavaScript.
The question is how much of the style comes with us.

This Lecture

Last week: loops as `for` forms, values on demand, and `struct`.

This week we leave Racket for Python and JavaScript.
The question is how much of the style comes with us.

Three things to get out of it:

1. **Reading** a closure precisely, down to the binding it captures.
2. **Choosing** between a collection, an iterator, and a generator.
3. **Recognising** the functional style under a language's own syntax.

This Lecture

Last week: loops as `for` forms, values on demand, and `struct`.

This week we leave Racket for Python and JavaScript.
The question is how much of the style comes with us.

Three things to get out of it:

1. **Reading** a closure precisely, down to the binding it captures.
2. **Choosing** between a collection, an iterator, and a generator.
3. **Recognising** the functional style under a language's own syntax.

Both languages check types at run time, as Racket does.
Static types wait for Haskell, next week.

One Query, Three Languages

The names of the students with a grade above four, in upper case:

Racket, a list of pairs

```
(map string-upcase
 (map car
  (filter
   (lambda (s) (> (cdr s) 4))
   students)))
```

Python, a list of tuples

```
list(map(
  lambda s: s[0].upper(),
  filter(
    lambda s: s[1] > 4,
    students)))
```

JavaScript, an array of objects

```
students
  .filter(s => s.grade > 4)
  .map(s => s.name)
  .map(name =>
    name.toUpperCase())
```

One Query, Three Languages

The names of the students with a grade above four, in upper case:

Racket, a list of pairs

```
(map string-upcase
 (map car
  (filter
   (lambda (s) (> (cdr s) 4))
   students)))
```

Python, a list of tuples

```
list(map(
  lambda s: s[0].upper(),
  filter(
    lambda s: s[1] > 4,
    students)))
```

JavaScript, an array of objects

```
students
  .filter(s => s.grade > 4)
  .map(s => s.name)
  .map(name =>
    name.toUpperCase())
```

The syntax differs.

The decomposition is the same: select, then transform.

One Query, Three Languages

The names of the students with a grade above four, in upper case:

Racket, a list of pairs

```
(map string-upcase
 (map car
  (filter
   (lambda (s) (> (cdr s) 4))
   students)))
```

Python, a list of tuples

```
list(map(
  lambda s: s[0].upper(),
  filter(
    lambda s: s[1] > 4,
    students)))
```

JavaScript, an array of objects

```
students
  .filter(s => s.grade > 4)
  .map(s => s.name)
  .map(name =>
    name.toUpperCase())
```

The syntax differs.
The decomposition is the same: select, then transform.

In Racket and Python the stages nest, so we read them inside out.
In JavaScript they chain, so we read them top down.

The Same Query as a Comprehension

Racket and Python also write the query as one loop over the list:

Racket

```
(for/list ([s students]
           #:when (> (cdr s) 4))
  (string-upcase (car s)))
```

Python

```
[name.upper()
 for name, grade in students
 if grade > 4]
```

The Same Query as a Comprehension

Racket and Python also write the query as one loop over the list:

Racket

```
(for/list ([s students]
           #:when (> (cdr s) 4))
  (string-upcase (car s)))
```

Python

```
[name.upper()
 for name, grade in students
 if grade > 4]
```

A **comprehension** is the select and the transform in one form.
In Python it is the idiomatic way to write a query over a list.

The Same Query as a Comprehension

Racket and Python also write the query as one loop over the list:

Racket

```
(for/list ([s students]
           #:when (> (cdr s) 4))
  (string-upcase (car s)))
```

Python

```
[name.upper()
 for name, grade in students
 if grade > 4]
```

A **comprehension** is the select and the transform in one form.
In Python it is the idiomatic way to write a query over a list.

JavaScript has no comprehensions.
The chain of array methods is its idiom.

What Comes With Us

A language is not functional because it has a `map`.
The style is in how we write:

[1] J. Hughes (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2. URL: https://doi.org/10.1093/comjnl/32.2.98.

What Comes With Us

A language is not functional because it has a `map`.
The style is in how we write:

- we compute new values from old ones, and do not change the old ones
- we build a program from small functions, and compose them
- effects (input, output, time, randomness) stay in one small place

[1] J. Hughes (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2. URL: https://doi.org/10.1093/comjnl/32.2.98.

What Comes With Us

A language is not functional because it has a `map`.
The style is in how we write:

- we compute new values from old ones, and do not change the old ones
- we build a program from small functions, and compose them
- effects (input, output, time, randomness) stay in one small place

Hughes's two kinds of glue are higher-order functions and lazy evaluation[1].
Both are available in Python and JavaScript, as we will see.

[1] J. Hughes (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2. URL: https://doi.org/10.1093/comjnl/32.2.98.

What Comes With Us

A language is not functional because it has a `map`.
The style is in how we write:

- we compute new values from old ones, and do not change the old ones
- we build a program from small functions, and compose them
- effects (input, output, time, randomness) stay in one small place

Hughes's two kinds of glue are higher-order functions and lazy evaluation[1].
Both are available in Python and JavaScript, as we will see.

Neither language enforces any of this.
Thus, the style is a set of choices, made one function at a time.

[1] J. Hughes (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2. URL: https://doi.org/10.1093/comjnl/32.2.98.

Pure Functions and Mutation

Pure Functions

A function is **pure** when its result depends on its arguments only, and the call changes nothing outside it.

```
def line_total(price, quantity):
    return price * quantity
```

Pure Functions

A function is **pure** when its result depends on its arguments only, and the call changes nothing outside it.

```
def line_total(price, quantity):
    return price * quantity
```

For the same arguments, a call may be replaced by its value.
The substitution model applies, so we can read one expression at a time.

```
line_total(80, 2) + line_total(80, 2)    # reads as 160 + 160
```

Pure Functions

A function is **pure** when its result depends on its arguments only, and the call changes nothing outside it.

```
def line_total(price, quantity):
    return price * quantity
```

For the same arguments, a call may be replaced by its value.
The substitution model applies, so we can read one expression at a time.

```
line_total(80, 2) + line_total(80, 2)    # reads as 160 + 160
```

Reading a global, printing, changing an argument, the clock, or a random number breaks the replacement.

`map` Does Not Make the Callback Pure

What does this program print?

```
let calls = 0;
const doubled = [1, 2, 3].map(x => {
  calls += 1;
  return 2 * x;
});
console.log(calls, doubled);
```

`map` Does Not Make the Callback Pure

What does this program print?

```
let calls = 0;
const doubled = [1, 2, 3].map(x => {
  calls += 1;
  return 2 * x;
});
console.log(calls, doubled);
```

It prints `3` and `[2, 4, 6]`.
The callback returns a value, and it also changes `calls`.

`map` Does Not Make the Callback Pure

What does this program print?

```
let calls = 0;
const doubled = [1, 2, 3].map(x => {
  calls += 1;
  return 2 * x;
});
console.log(calls, doubled);
```

It prints `3` and `[2, 4, 6]`.
The callback returns a value, and it also changes `calls`.

`map` decides the traversal.
The callback decides whether the computation is pure.
The same holds for `map` in Racket and `map` in Python.

`const` Fixes the Name, Not the Value

In JavaScript, **`const`** stops the name from being reassigned:

```
const grades = [4, 5];
grades.push(3);        // fine: the array changes
// grades = [5, 5];    // TypeError: the name cannot
```

Likewise, a Python tuple may hold a list:

```
record = ("Anna", [4, 5])
record[1].append(3)   # fine: the inner list changes
```

[2] MDN contributors (2026b). ***const***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const.

`const` Fixes the Name, Not the Value

In JavaScript, **`const`** stops the name from being reassigned:

```
const grades = [4, 5];
grades.push(3);        // fine: the array changes
// grades = [5, 5];    // TypeError: the name cannot
```

Likewise, a Python tuple may hold a list:

```
record = ("Anna", [4, 5])
record[1].append(3)   # fine: the inner list changes
```

A binding and a value are different things.
`const` fixes the binding, and the array behind it still changes[2].

[2] MDN contributors (2026b). ***const***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const.

`const` Fixes the Name, Not the Value

In JavaScript, **`const`** stops the name from being reassigned:

```
const grades = [4, 5];
grades.push(3);        // fine: the array changes
// grades = [5, 5];    // TypeError: the name cannot
```

Likewise, a Python tuple may hold a list:

```
record = ("Anna", [4, 5])
record[1].append(3)   # fine: the inner list changes
```

A binding and a value are different things.
`const` fixes the binding, and the array behind it still changes[2].

Thus, a value stays immutable only because we do not mutate it.

[2] MDN contributors (2026b). ***const***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const.

A Copy Is Shallow

A comprehension, and JavaScript's `map`, make a new outer collection. They do not copy the elements.

```
students = [{"name": "Anna", "grades": [4, 5]}]
same_students = [student for student in students]
same_students[0]["grades"].append(3)
students[0]["grades"]       # [4, 5, 3]
```

A Copy Is Shallow



A comprehension, and JavaScript's `map`, make a new outer collection.
They do not copy the elements.

```
students = [{"name": "Anna", "grades": [4, 5]}]
same_students = [student for student in students]
same_students[0]["grades"].append(3)
students[0]["grades"]       # [4, 5, 3]
```

The two lists are different.
Their first elements are the same dictionary.

A Copy Is Shallow

A comprehension, and JavaScript's `map`, make a new outer collection.
They do not copy the elements.

```
students = [{"name": "Anna", "grades": [4, 5]}]
same_students = [student for student in students]
same_students[0]["grades"].append(3)
students[0]["grades"]       # [4, 5, 3]
```

The two lists are different.
Their first elements are the same dictionary.

Thus, a pipeline never changes the list it walks.
However, the elements are shared, and a change to one shows in both lists.

Read a Pipeline One Stage at a Time

Read a chain from its source, one stage at a time:

```
const result = students
  .filter(student => student.grade > 4)
  .map(student => student.name)
  .map(name => name.toUpperCase());
```

Read a Pipeline One Stage at a Time

Read a chain from its source, one stage at a time:

```
const result = students
  .filter(student => student.grade > 4)
  .map(student => student.name)
  .map(name => name.toUpperCase());
```

1. `filter`: keep the students with a grade above four.

Read a Pipeline One Stage at a Time

Read a chain from its source, one stage at a time:

```
const result = students
  .filter(student => student.grade > 4)
  .map(student => student.name)
  .map(name => name.toUpperCase());
```

1. `filter`: keep the students with a grade above four.
2. The first `map`: replace each student by a name.

Read a Pipeline One Stage at a Time



Read a chain from its source, one stage at a time:

```
const result = students
  .filter(student => student.grade > 4)
  .map(student => student.name)
  .map(name => name.toUpperCase());
```

1. `filter`: keep the students with a grade above four.
2. The first `map`: replace each student by a name.
3. The second `map`: replace each name by its upper-case form.

Read a Pipeline One Stage at a Time

Read a chain from its source, one stage at a time:

```
const result = students
  .filter(student => student.grade > 4)
  .map(student => student.name)
  .map(name => name.toUpperCase());
```

1. `filter`: keep the students with a grade above four.
2. The first `map`: replace each student by a name.
3. The second `map`: replace each name by its upper-case form.

Each stage has one job, and its result has a shape we can name.
Note that one callback doing all three is harder to read than a loop.

Folds: Which Argument Is the Accumulator

The same total in the three languages:

Racket: the element first

```
(foldl
 (lambda (price total)
   (+ total price))
 0 prices)
```

Python: the accumulator first

```
from functools import reduce
reduce(
  lambda total, price:
    total + price,
  prices, 0)
```

JavaScript: the accumulator first

```
prices.reduce(
  (total, price) =>
    total + price,
  0)
```

Folds: Which Argument Is the Accumulator

The same total in the three languages:

Racket: the element first

```
(foldl
 (lambda (price total)
   (+ total price))
 0 prices)
```

Python: the accumulator first

```
from functools import reduce
reduce(
  lambda total, price:
    total + price,
  prices, 0)
```

JavaScript: the accumulator first

```
prices.reduce(
  (total, price) =>
    total + price,
  0)
```

Racket passes the element first, Python and JavaScript the accumulator first. The initial value is the result for an empty input, so we write it every time.

Closures

A Function That Needs an Environment

```
(define (less-than limit)
  (lambda (x) (< x limit)))

(define cheap? (less-than 100))
(filter cheap? '(80 120 95 200 60))
; '(80 95 60)
```

A Function That Needs an Environment

```
(define (less-than limit)
  (lambda (x) (< x limit)))

(define cheap? (less-than 100))
(filter cheap? '(80 120 95 200 60))
; '(80 95 60)
```

The returned function mentions `limit`, but its only parameter is `x`.
The name `limit` is **free** in the **`lambda`**.

A Function That Needs an Environment



```
(define (less-than limit)
  (lambda (x) (< x limit)))

(define cheap? (less-than 100))
(filter cheap? '(80 120 95 200 60))
; '(80 95 60)
```

The returned function mentions `limit`, but its only parameter is `x`.
The name `limit` is **free** in the **`lambda`**.

A **closure** is the code of a function together with the environment that binds its free names.
Here the environment binds `limit` to `100`.

The Factory Pattern

The same closure in Python and JavaScript:

```python
def less_than(limit):
    def predicate(x):
        return x < limit
    return predicate

cheap = less_than(100)
```

```javascript
function lessThan(limit) {
  return x => x < limit;
}

const cheap = lessThan(100);
```

The Factory Pattern

The same closure in Python and JavaScript:

```python
def less_than(limit):
    def predicate(x):
        return x < limit
    return predicate

cheap = less_than(100)
```

```javascript
function lessThan(limit) {
  return x => x < limit;
}

const cheap = lessThan(100);
```

Each call to the outer function creates a fresh binding of `limit`.
The returned closure keeps that binding alive after the call has returned.

The Factory Pattern

The same closure in Python and JavaScript:

```python
def less_than(limit):
    def predicate(x):
        return x < limit
    return predicate

cheap = less_than(100)
```

```javascript
function lessThan(limit) {
  return x => x < limit;
}

const cheap = lessThan(100);
```

Each call to the outer function creates a fresh binding of `limit`.
The returned closure keeps that binding alive after the call has returned.

Thus, a closure lets us fix part of the input now and supply the rest later.

A Closure Captures a Binding, Not a Value

```
limit = 100
cheap = lambda price: price < limit
cheap(80)     # True
limit = 50
cheap(80)     # False
```

A Closure Captures a Binding, Not a Value

```
limit = 100
cheap = lambda price: price < limit
cheap(80)     # True
limit = 50
cheap(80)     # False
```

The closure looks the binding up when it is called, not when it is created.
If the binding is assigned in between, the answer changes.

A Closure Captures a Binding, Not a Value

```
limit = 100
cheap = lambda price: price < limit
cheap(80)     # True
limit = 50
cheap(80)     # False
```

The closure looks the binding up when it is called, not when it is created.
If the binding is assigned in between, the answer changes.

A factory's parameter is never assigned again.
That is why the factory pattern is easy to reason about.

A Closure With State

A closure may keep private state on purpose:

```
def make_counter():
    count = 0
    def next_count():
        nonlocal count
        count += 1
        return count
    return next_count
c = make_counter()
c(), c(), c()       # (1, 2, 3)
```

A Closure With State

A closure may keep private state on purpose:

```
def make_counter():
    count = 0
    def next_count():
        nonlocal count
        count += 1
        return count
    return next_count
c = make_counter()
c(), c(), c()       # (1, 2, 3)
```

With `nonlocal`, `count` is the enclosing function's binding, not a new one. Nothing but `next_count` can reach it.

A Closure With State

A closure may keep private state on purpose:

```
def make_counter():
    count = 0
    def next_count():
        nonlocal count
        count += 1
        return count
    return next_count
c = make_counter()
c(), c(), c()       # (1, 2, 3)
```

With **`nonlocal`**, `count` is the enclosing function's binding, not a new one. Nothing but `next_count` can reach it.

However, `next_count` is not pure, since equal calls return different values.

Question: Which Binding?

```
limit = 100
def make_check(limit):
    return lambda price: price < limit
cheap = make_check(50)
limit = 10
cheap(40)
```

Question: Which Binding?

```
limit = 100
def make_check(limit):
    return lambda price: price < limit
cheap = make_check(50)
limit = 10
cheap(40)
```

The value is **`True`**.

Question: Which Binding?

```
limit = 100
def make_check(limit):
    return lambda price: price < limit
cheap = make_check(50)
limit = 10
cheap(40)
```

The value is **`True`**.

The **`lambda`** is written inside `make_check`.
Its `limit` is the parameter binding, whose value is `50`.
The later assignment changes the top-level binding, which is a different one.

Question: Which Binding?

```
limit = 100
def make_check(limit):
    return lambda price: price < limit
cheap = make_check(50)
limit = 10
cheap(40)
```

The value is **`True`**.

The **`lambda`** is written inside `make_check`.
Its `limit` is the parameter binding, whose value is `50`.
The later assignment changes the top-level binding, which is a different one.

Where the **`lambda`** is written decides which binding `limit` means.
What has happened to that binding since decides the value.

5 min break



Python

Collections, Iterables, and Iterators

Python separates three roles:

- A **collection**, such as a list, stores its values and can be walked again.
- An **iterable** produces an iterator, with `iter(value)`.
- An **iterator** produces the next value, with `next(iterator)`.

```
values = [10, 20, 30]
it = iter(values)
next(it), next(it)       # (10, 20)
list(it)                 # [30]
list(it)                 # []
```

[3] Python Software Foundation (2026). ***Functional Programming HOWTO.*** URL: https://docs.python.org/3/howto/functional.html.

Collections, Iterables, and Iterators

Python separates three roles:

- A **collection**, such as a list, stores its values and can be walked again.
- An **iterable** produces an iterator, with `iter(value)`.
- An **iterator** produces the next value, with `next(iterator)`.

```
values = [10, 20, 30]
it = iter(values)
next(it), next(it)       # (10, 20)
list(it)                 # [30]
list(it)                 # []
```

An iterator has state, and it is used up once[3].

A **`for`** loop calls `iter` on its collection, then `next` until the iterator is exhausted.

[3] Python Software Foundation (2026). ***Functional Programming HOWTO***. URL: https://docs.python.org/3/howto/functional.html.

Generators: `yield`

A generator function is written with **`yield`**, as in Racket:

```
def squares_of_evens(values):
    for x in values:
        if x % 2 == 0:
            yield x * x

it = squares_of_evens([1, 2, 3, 4])
next(it)    # 4
next(it)    # 16
list(it)    # []
```

Generators: `yield`

A generator function is written with **`yield`**, as in Racket:

```
def squares_of_evens(values):
    for x in values:
        if x % 2 == 0:
            yield x * x

it = squares_of_evens([1, 2, 3, 4])
next(it)    # 4
next(it)    # 16
list(it)    # []
```

The call runs nothing.

It returns a generator, an iterator whose `next` runs the body to the next **`yield`**.

Generators: `yield`

A generator function is written with **`yield`**, as in Racket:

```
def squares_of_evens(values):
    for x in values:
        if x % 2 == 0:
            yield x * x

it = squares_of_evens([1, 2, 3, 4])
next(it)    # 4
next(it)    # 16
list(it)    # []
```

```
def naturals(n):
    while True:
        yield n
        n += 1

it = naturals(0)
next(it)    # 0
next(it)    # 1
```

The call runs nothing.

It returns a generator, an iterator whose `next` runs the body to the next **`yield`**.

Thus, the values may have no end, since nothing runs ahead of the calls.

Eager and Incremental Python

Square the even values:

```
eager = [x * x for x in values if x % 2 == 0]
incremental = (x * x for x in values if x % 2 == 0)
type(eager)        # list
type(incremental)  # generator
```

Eager and Incremental Python

Square the even values:

```
eager = [x * x for x in values if x % 2 == 0]
incremental = (x * x for x in values if x % 2 == 0)
type(eager)       # list
type(incremental) # generator
```

Square brackets build the whole list at once.
Parentheses make a **generator expression**, one value per request.

Eager and Incremental Python

Square the even values:

```
eager = [x * x for x in values if x % 2 == 0]
incremental = (x * x for x in values if x % 2 == 0)
type(eager)        # list
type(incremental)  # generator
```

Square brackets build the whole list at once.
Parentheses make a **generator expression**, one value per request.

`map` and `filter` return iterators too.
We call `list` only when the whole list is needed.

Eager and Incremental Python

Square the even values:

```
eager = [x * x for x in values if x % 2 == 0]
incremental = (x * x for x in values if x % 2 == 0)
type(eager)        # list
type(incremental)  # generator
```

Square brackets build the whole list at once.
Parentheses make a **generator expression**, one value per request.

`map` and `filter` return iterators too.
We call `list` only when the whole list is needed.

If `values` had no end, the generator would still work, and the list would not.

The One-Shot Trap

What do the four calls return?

```
squares = (x * x for x in [1, 2, 3, 4])
next(squares)
list(squares)

upper = map(str.upper, ["a", "b"])
list(upper)
list(upper)
```

The One-Shot Trap

What do the four calls return?

```
squares = (x * x for x in [1, 2, 3, 4])
next(squares)
list(squares)

upper = map(str.upper, ["a", "b"])
list(upper)
list(upper)
```

`1`, then `[4, 9, 16]`: `next` takes one value, and `list` the rest.

The One-Shot Trap

What do the four calls return?

```
squares = (x * x for x in [1, 2, 3, 4])
next(squares)
list(squares)

upper = map(str.upper, ["a", "b"])
list(upper)
list(upper)
```

`1`, then `[4, 9, 16]`: `next` takes one value, and `list` the rest.

`['A', 'B']`, then `[]`: the `map` object is an iterator, used up by the first `list`.

The One-Shot Trap

What do the four calls return?

```
squares = (x * x for x in [1, 2, 3, 4])
next(squares)
list(squares)

upper = map(str.upper, ["a", "b"])
list(upper)
list(upper)
```

`1`, then `[4, 9, 16]`: `next` takes one value, and `list` the rest.

`['A', 'B']`, then `[]`: the `map` object is an iterator, used up by the first `list`.

Thus, a result we need twice has to be stored, with `list`, before it is read.

A Python Pipeline

The first three upper-case names with a grade above four:

```
from itertools import islice

names = (name.upper()
         for name, grade in students
         if grade > 4)
first_three = list(islice(names, 3))
```

A Python Pipeline

The first three upper-case names with a grade above four:

```
from itertools import islice

names = (name.upper()
         for name, grade in students
         if grade > 4)
first_three = list(islice(names, 3))
```

The generator expression selects and transforms in one place.
`islice` stops after three values, and `list` stores those three.

A Python Pipeline

The first three upper-case names with a grade above four:

```
from itertools import islice

names = (name.upper()
         for name, grade in students
         if grade > 4)
first_three = list(islice(names, 3))
```

The generator expression selects and transforms in one place.
`islice` stops after three values, and `list` stores those three.

A comprehension usually reads better than nested `map` and `filter`.
The stages are still there, and we still name them.

The Loop-Closure Trap

What is the value of the last line?

```
adders = []
for offset in range(3):
    adders.append(lambda x: x + offset)
[add(10) for add in adders]
```

The Loop-Closure Trap

What is the value of the last line?

```
adders = []
for offset in range(3):
    adders.append(lambda x: x + offset)
[add(10) for add in adders]
```

It is `[12, 12, 12]`.

The Loop-Closure Trap

What is the value of the last line?

```
adders = []
for offset in range(3):
    adders.append(lambda x: x + offset)
[add(10) for add in adders]
```

It is `[12, 12, 12]`.

The loop has one binding of `offset`, assigned three times.
Each closure reads it after the loop, when its value is `2`.

The Loop-Closure Trap: Two Fixes

Make the binding explicit with a factory:

```
def add_offset(offset):
    return lambda x: x + offset
adders = [add_offset(offset) for offset in range(3)]
[add(10) for add in adders]      # [10, 11, 12]
```

The Loop-Closure Trap: Two Fixes

Make the binding explicit with a factory:

```
def add_offset(offset):
    return lambda x: x + offset
adders = [add_offset(offset) for offset in range(3)]
[add(10) for add in adders]      # [10, 11, 12]
```

Each call to the factory creates a fresh binding of `offset`.

The Loop-Closure Trap: Two Fixes

Make the binding explicit with a factory:

```
def add_offset(offset):
    return lambda x: x + offset
adders = [add_offset(offset) for offset in range(3)]
[add(10) for add in adders]      # [10, 11, 12]
```

Each call to the factory creates a fresh binding of `offset`.

A default argument works too, since it copies the value at definition:

```
lambda x, offset=offset: x + offset
```

However, the factory states the capture, and the default argument hides it.

Recursion Is Not the Default in Python

```python
def total(xs, acc=0):
    if not xs:
        return acc
    return total(xs[1:], acc + xs[0])
```

Recursion Is Not the Default in Python

```
def total(xs, acc=0):
    if not xs:
        return acc
    return total(xs[1:], acc + xs[0])
```

Python does not eliminate tail calls, so each element costs a stack frame. At about a thousand elements it raises `RecursionError`.

Recursion Is Not the Default in Python

```
def total(xs, acc=0):
    if not xs:
        return acc
    return total(xs[1:], acc + xs[0])
```

Python does not eliminate tail calls, so each element costs a stack frame. At about a thousand elements it raises `RecursionError`.

In Python we write `sum(xs)`, a loop, a comprehension, or a generator. Recursion stays for recursive data, such as a tree of bounded depth.

Recursion Is Not the Default in Python

```
def total(xs, acc=0):
    if not xs:
        return acc
    return total(xs[1:], acc + xs[0])
```

Python does not eliminate tail calls, so each element costs a stack frame. At about a thousand elements it raises `RecursionError`.

In Python we write `sum(xs)`, a loop, a comprehension, or a generator. Recursion stays for recursive data, such as a tree of bounded depth.

Functional programming does not mean replacing every loop by recursion.

JavaScript

Array Pipelines Are Eager

```
const result = values
  .filter(x => x % 2 === 0)
  .map(x => x * x);
```

[4] MDN contributors (2026a). ***Array***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array.

Array Pipelines Are Eager

```
const result = values
  .filter(x => x % 2 === 0)
  .map(x => x * x);
```

On an array, `filter` walks the whole input and builds an array. Then `map` walks that array and builds another one[4].

[4] MDN contributors (2026a). ***Array***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array.

Array Pipelines Are Eager

```
const result = values
  .filter(x => x % 2 === 0)
  .map(x => x * x);
```

On an array, `filter` walks the whole input and builds an array. Then `map` walks that array and builds another one[4].

The code is clear, and at ordinary sizes the extra arrays cost little. However, a chain of array methods is not incremental.

[4] MDN contributors (2026a). ***Array***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array.

Array Pipelines Are Eager

```
const result = values
  .filter(x => x % 2 === 0)
  .map(x => x * x);
```

On an array, `filter` walks the whole input and builds an array.
Then `map` walks that array and builds another one[4].

The code is clear, and at ordinary sizes the extra arrays cost little.
However, a chain of array methods is not incremental.

Use `find`, `some`, or `every` when the answer may be known before the end.
They stop at the first element that decides it.

[4] MDN contributors (2026a). ***Array***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array.

Generators Make JavaScript Incremental

```
function* squaresOfEvens(values) {
  for (const x of values) {
    if (x % 2 === 0) yield x * x;
  }
}

const it = squaresOfEvens([1, 2, 3, 4]);
it.next();  // { value: 4, done: false }
it.next();  // { value: 16, done: false }
```

Generators Make JavaScript Incremental

```
function* squaresOfEvens(values) {
  for (const x of values) {
    if (x % 2 === 0) yield x * x;
  }
}

const it = squaresOfEvens([1, 2, 3, 4]);
it.next();  // { value: 4, done: false }
it.next();  // { value: 16, done: false }
```

The star marks a generator function.

A call returns an iterator, and each `next()` runs the body to the next **`yield`**.

Generators Make JavaScript Incremental

```
function* squaresOfEvens(values) {
  for (const x of values) {
    if (x % 2 === 0) yield x * x;
  }
}

const it = squaresOfEvens([1, 2, 3, 4]);
it.next();  // { value: 4, done: false }
it.next();  // { value: 16, done: false }
```

The star marks a generator function.
A call returns an iterator, and each `next()` runs the body to the next **`yield`**.

The iterator is used up once.
A second call to `squaresOfEvens` starts a fresh traversal.

Loop Bindings: `var` and `let`



The loop-closure trap again, in JavaScript:

```
const oldAdders = [];
for (var i = 0; i < 3; i++) oldAdders.push(x => x + i);
oldAdders.map(f => f(10));       // [13, 13, 13]
```

[5] Ecma International (2026). ***ECMAScript 2026 Language Specification.*** URL: https://tc39.es/ecma262/2026/.

Loop Bindings: `var` and `let`



The loop-closure trap again, in JavaScript:

```
const oldAdders = [];
for (var i = 0; i < 3; i++) oldAdders.push(x => x + i);
oldAdders.map(f => f(10));       // [13, 13, 13]
```

`var` makes one binding for the whole function.
Every closure reads it after the loop, when its value is `3`.

[5] Ecma International (2026). ***ECMAScript 2026 Language Specification.*** URL: https://tc39.es/ecma262/2026/.

Loop Bindings: `var` and `let`

The loop-closure trap again, in JavaScript:

```
const oldAdders = [];
for (var i = 0; i < 3; i++) oldAdders.push(x => x + i);
oldAdders.map(f => f(10));        // [13, 13, 13]
```

`var` makes one binding for the whole function.
Every closure reads it after the loop, when its value is `3`.

```
const adders = [];
for (let j = 0; j < 3; j++) adders.push(x => x + j);
adders.map(f => f(10));           // [10, 11, 12]
```

A **`for`** loop with **`let`** makes a fresh binding for each iteration[5].

[5] Ecma International (2026). ***ECMAScript 2026 Language Specification.*** URL: https://tc39.es/ecma262/2026/.

Loop Bindings: `var` and `let`

The loop-closure trap again, in JavaScript:

```
const oldAdders = [];
for (var i = 0; i < 3; i++) oldAdders.push(x => x + i);
oldAdders.map(f => f(10));        // [13, 13, 13]
```

`var` makes one binding for the whole function.
Every closure reads it after the loop, when its value is `3`.

```
const adders = [];
for (let j = 0; j < 3; j++) adders.push(x => x + j);
adders.map(f => f(10));           // [10, 11, 12]
```

A **`for`** loop with **`let`** makes a fresh binding for each iteration[5].

In modern JavaScript we write **`const`** and **`let`**, and not **`var`**.

[5] Ecma International (2026). ***ECMAScript 2026 Language Specification.*** URL: https://tc39.es/ecma262/2026/.

Non-Mutating Array Operations

Many familiar methods make a new array and leave the old one alone. Among them are `map`, `filter`, `slice`, `concat`, `toSorted`, and `toReversed`.

```
const values = [3, 1, 2];
const sorted = values.toSorted();
values;  // [3, 1, 2]
sorted;  // [1, 2, 3]
```

Non-Mutating Array Operations

Many familiar methods make a new array and leave the old one alone.
Among them are `map`, `filter`, `slice`, `concat`, `toSorted`, and `toReversed`.

```
const values = [3, 1, 2];
const sorted = values.toSorted();
values;  // [3, 1, 2]
sorted;  // [1, 2, 3]
```

Their copies are shallow, as before.
In contrast, `sort`, `reverse`, `push`, and `splice` change the array in place.

Choosing a Representation

The Same Ideas, Different Defaults

	Racket	Python	JavaScript
Ordinary collection	immutable list	mutable list	mutable array
Select and transform	`filter`, `map`	comprehension	`filter`, `map`
Values on demand	stream, generator	generator, iterator	generator, iterator
`map` and `filter`	eager, on lists	incremental	eager, on arrays
Loop variable	fresh per iteration	one, assigned	fresh with `let`, one with `var`
Tail calls	safe	not safe	not safe in practice
Purity	by discipline	by discipline	by discipline

The Same Ideas, Different Defaults

	Racket	Python	JavaScript
Ordinary collection	immutable list	mutable list	mutable array
Select and transform	`filter`, `map`	comprehension	`filter`, `map`
Values on demand	stream, generator	generator, iterator	generator, iterator
`map` and `filter`	eager, on lists	incremental	eager, on arrays
Loop variable	fresh per iteration	one, assigned	fresh with `let`, one with `var`
Tail calls	safe	not safe	not safe in practice
Purity	by discipline	by discipline	by discipline

The same shape of code may evaluate differently in the three languages.
Ask what is stored, what is computed on demand, and where the state lives.

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

1. Read input, time, randomness, or the network at the boundary.

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

1. Read input, time, randomness, or the network at the boundary.
2. Turn them into plain values.

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

1. Read input, time, randomness, or the network at the boundary.
2. Turn them into plain values.
3. Transform the values with small, pure functions.

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

1. Read input, time, randomness, or the network at the boundary.
2. Turn them into plain values.
3. Transform the values with small, pure functions.
4. Print, store, or send the result at the boundary.

A Functional Core and an Effectful Boundary

One way to organise a program, in any of the three languages:

1. Read input, time, randomness, or the network at the boundary.
2. Turn them into plain values.
3. Transform the values with small, pure functions.
4. Print, store, or send the result at the boundary.

The effects do not go away.
They are gathered where we can see them and test around them.

When a Loop Is Clearer

A pipeline is the right choice when each stage has a clear input and output, and the library operation says what we mean.

When a Loop Is Clearer

A pipeline is the right choice when each stage has a clear input and output, and the library operation says what we mean.

A named helper or a plain loop is clearer when

- a callback grows several branches or effects
- a chain copies a growing accumulator at every step
- the intermediate values are harder to name than the state

When a Loop Is Clearer

A pipeline is the right choice when each stage has a clear input and output, and the library operation says what we mean.

A named helper or a plain loop is clearer when

- a callback grows several branches or effects
- a chain copies a growing accumulator at every step
- the intermediate values are harder to name than the state

The point of the style is that each piece can be read on its own.
Shorter code that loses this is not an improvement.

Quick Recap

1. What makes a function pure?
2. What does a closure contain?
3. Is a JavaScript array named by `const` immutable?
4. A Python list comprehension against a generator expression?
5. Is a chain of JavaScript array methods incremental?

Quick Recap

1. What makes a function pure?
   Its result depends on its arguments only, and the call changes nothing outside it.
2. What does a closure contain?
3. Is a JavaScript array named by `const` immutable?
4. A Python list comprehension against a generator expression?
5. Is a chain of JavaScript array methods incremental?

Quick Recap

1. What makes a function pure?
   Its result depends on its arguments only, and the call changes nothing outside it.
2. What does a closure contain?
   The code of a function and the environment that binds its free names.
3. Is a JavaScript array named by `const` immutable?
4. A Python list comprehension against a generator expression?
5. Is a chain of JavaScript array methods incremental?

Quick Recap

1. What makes a function pure?
   Its result depends on its arguments only, and the call changes nothing outside it.
2. What does a closure contain?
   The code of a function and the environment that binds its free names.
3. Is a JavaScript array named by `const` immutable?
   No, only the name is fixed, and the array may still change.
4. A Python list comprehension against a generator expression?

5. Is a chain of JavaScript array methods incremental?

Quick Recap

1. What makes a function pure?
   Its result depends on its arguments only, and the call changes nothing outside it.
2. What does a closure contain?
   The code of a function and the environment that binds its free names.
3. Is a JavaScript array named by `const` immutable?
   No, only the name is fixed, and the array may still change.
4. A Python list comprehension against a generator expression?
   The first builds the whole list, the second computes a value when asked.
5. Is a chain of JavaScript array methods incremental?

Quick Recap

1. What makes a function pure?
   Its result depends on its arguments only, and the call changes nothing outside it.
2. What does a closure contain?
   The code of a function and the environment that binds its free names.
3. Is a JavaScript array named by `const` immutable?
   No, only the name is fixed, and the array may still change.
4. A Python list comprehension against a generator expression?
   The first builds the whole list, the second computes a value when asked.
5. Is a chain of JavaScript array methods incremental?
   No, each method builds its whole array before the next one starts.

Quiz for Lecture 5

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 5

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

> Say what you learned and what stayed unclear, **specifically**.
> I read them before writing the next lecture.

What's Next



In today's lab you will

- write the worksheet on this lecture
- defend problem set 4, if your name is drawn
- work on mini project 1, due 28 September

→ **Before next week: Haskell**

- There is no problem set for Topic 5. The week is the project's.
- Install GHCup from `https://www.haskell.org/ghcup/`.
- Read *Learn You a Haskell for Great Good*, Chapters 2 and 4, at `https://learnyouahaskell.com/chapters`.

Thank you!



References

- Ecma International (2026). ***ECMAScript 2026 Language Specification***. URL: https://tc39.es/ecma262/2026/ (visited on 09/16/2026).
- Hughes, J. (Jan. 1989). **"Why Functional Programming Matters"**. In: *The Computer Journal* 32.2, pp. 98–107. ISSN: 0010-4620. DOI: 10.1093/comjnl/32.2.98. eprint: https://academic.oup.com/comjnl/article-pdf/32/2/98/1445644/320098.pdf. URL: https://doi.org/10.1093/comjnl/32.2.98.
- MDN contributors (2026a). ***Array***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array (visited on 09/16/2026).

References ii

- MDN contributors (2026b). ***const***. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const (visited on 09/16/2026).
- Python Software Foundation (2026). ***Functional Programming HOWTO***. URL: https://docs.python.org/3/howto/functional.html (visited on 09/16/2026).